

南京大学本科生实验报告

课程名称：计算机网络 任课教师：黄程远 助教：刘松岳

学院	计算机学院	专业（方向）	计算机金融工程实验班
学号	221275014	姓名	赵子函
Email	2337951984@qq.com	开始/完成日期	2024.9.14/2024.9.24

一、实验名称

Lab 4: Respond to ARP

二、实验目的

实现路由器部分功能：构建转发表并进行目的地址匹配，发送 ARP 请求并转发数据报

三、实验内容

1.Task 1: Preparation

用 git clone 克隆代码

2.Task 2: IP Forwarding Table Lookup; Task 3: Forwarding the Packet and ARP

构建一个转发表，匹配目标地址；
发送 ARP 查询以创建新的以太网标头并转发数据包

（1）核心代码

①主要类与函数

■ 类 ARPPacket

```
class ARPPacket(object):
    def __init__(self,ip_packet,interface):
        self.ip_packet=ip_packet
        self.interface=interface
        self.sent_time=time.time()
        self.retries=0
```

- 功能： 用于表示一个等待 ARP 回复的 IPv4 数据包。
- 属性：
 - `ip\_packet`：待发送的 IP 数据包。
 - `interface`：发送该 IP 数据包的接口。
 - `sent\_time`：发送时间戳。
 - `retries`：重试次数。
- 作用： 存储与 ARP 请求相关的信息，如待发送的 IP 数据包、发送接口、发送时间和重试次数。

■ 类 Router

- 功能： 表示一个基本的 IPv4 静态路由器。
- 属性：
 - `net`： 网络底层接口。
 - `intf\_ipaddrs`： 路由器接口的 IP 地址列表。
 - `forwarding\_table`： 转发表。
 - `sorted\_forwarding\_table`： 按前缀长度排序的转发表。
 - `arp\_table`： ARP 缓存表。
 - `waiting\_list`： 等待 ARP 解析的数据包队列。
- 方法：
 - `\_\_init\_\_`： 初始化路由器，构建接口 IP 地址列表、转发表、ARP 缓存表和等待列表。
 - `match`： 在转发表中查找匹配目标地址的网络条目。
 - `send\_arp\_request`： 发送 ARP 请求。
 - `send\_arp\_reply`： 发送 ARP 回复。
 - `append\_waiting\_list`： 将数据包添加到等待列表中。
 - `update\_arp\_cache`： 更新 ARP 缓存表并发送等待队列中的数据包。
 - `forward\_packet`： 转发数据包。
 - `handle\_packet`： 处理接收到的数据包。
 - `check`： 检查等待列表中的 ARP 请求是否超时。
 - `start`： 开始运行路由器守护进程。
 - `stop`： 停止路由器。

■ 函数 main

- 功能： 程序的入口点，创建 Router 对象并启动它。

②代码主要逻辑

■ 路由器类 (Router) 的初始化

```
def __init__(self, net: switchyard.llnetbase.LLNetBase):  
    self.net = net  
  
    # create a list storing the ethaddr of interfaces of the router  
    self.intf_ipaddrs = []  
  
    for interface in self.net.interfaces():  
        self.intf_ipaddrs.append(interface.ipaddr)  
  
    # create forwarding table  
    # the list of router interfaces  
    self.forwarding_table=[]  
  
    for interface in self.net.interfaces():  
        print(f"{interface.ipaddr}")  
        print(f"{interface.netmask}")  
        ipaddr = interface.ipaddr  
        netmask = interface.netmask  
        print(f"ipaddr{ipaddr} netmask{netmask} ")  
        network_addr = IPv4Address(int(ipaddr)&int(netmask))  
        print(f"network_addr:{network_addr}")
```

```

netaddr = IPv4Network(f"{network_addr}/{netmask}")
print("netaddr:{netaddr}")

self.forwarding_table.append({
    "network_addr":network_addr,
    "mask":netmask,
    "next_hop":"0.0.0.0",
    "interface": interface,
    "prefixlen":netaddr.prefixlen
})

```

- Router 类初始化时接收一个 switchyard.llnetbase.LLNetBase 网络对象，它代表该路由器所接入的网络。
- 路由器会遍历所有接口，提取每个接口的 IP 地址和子网掩码，将它们存储在 self.intf_ipaddrs 和 self.forwarding_table 中，形成初始的路由表项（包含网络地址、子网掩码、下一跳和接口信息）。

■ 路由表的构建

```

# read forwarding table file
with open('forwarding_table.txt','r') as f:
    lines=f.readlines()
for line in lines:
    # split the line
    parts=line.strip().split()
    network_addr=parts[0]
    subnet_mask=parts[1]
    next_hop=IPv4Address(parts[2])
    ifaceName=parts[3]
    intf = self.net.interface_by_name(ifaceName)
    # convert networkaddr and subnet mask to IPv4Network objects
    netaddr = IPv4Network(f"{network_addr}/{subnet_mask}")
    # add entry to forwarding table
    self.forwarding_table.append({
        "network_addr":network_addr,
        "mask":subnet_mask,
        "next_hop":next_hop,
        "interface": intf,
        "prefixlen":netaddr.prefixlen
    })

self.sorted_forwarding_table = sorted(self.forwarding_table,key=lambda
x:x['prefixlen'],reverse=True)

print("Routing Table:")
print("{:<20}{:<20}{:<20}{:<20}".format("Network Address","Mask","Next Hop","Interface"))
for item in self.sorted_forwarding_table:
    print("{:<20}{:<20}{:<20}{:<20}".format(str(item['network_addr']),str(item['mask']),str(i
tem['next_hop']),str(item['interface'].name)))

```

```
self.arp_table={} # create cache arp table

self.waiting_list={} # queue of ARP packets waiting for resolution
```

- 路由器会从 forwarding_table.txt 文件中读取静态路由表，解析每行信息并加入到 self.forwarding_table 中。
- 每个路由表条目包含网络地址、子网掩码、下一跳地址（next_hop）和出接口。
- 路由表根据前缀长度（掩码的位数）从大到小进行排序，以便精确匹配优先。

■ 数据包的匹配与转发

```
def match(self,dest_addr):

    for item in self.sorted_forwarding_table:

        prefix = IPv4Address(item["network_addr"])
        mask = IPv4Address(item["mask"])
        dest_ip = IPv4Address(dest_addr)

        # print(f"prefix:{prefix} mask:{mask} dest_ip{dest_ip}\n{int(mask)&int(dest_ip)}
{int(prefix)} ")

        matches = (int(mask)&int(dest_ip))==int(prefix)

        if matches == True:

            return item["next_hop"],item["interface"]

    return None
```

- match 函数用于在路由表中查找与目的地址匹配的条目。
- 路由器将目的 IP 地址与表中每个条目的网络地址和掩码进行 AND 运算，匹配成功则返回下一跳地址和接口。

```
def forward_packet(self,packet):

    ip_header = packet.get_header(IPv4)
    item = self.match(ip_header.dst)

    if item is not None:# it must be not None,since it has been examine before
        next_hop,intf = item
        print("match successfully!")
        next_hop,intf = self.match(ip_header.dst)
        print(f"next_hop:{next_hop},intf:{intf}")
        if next_hop == "0.0.0.0":
            # print("next_hop is 0.0.0.0")
            macaddr = self.arp_table.get(ip_header.dst)
            if macaddr is not None:
                # print("get macaddr!")
                # print(macaddr)
                new_eth = Ethernet()
                new_eth.dst= macaddr
                new_eth.src= intf.ethaddr
                new_eth.ethertype=EtherType.IPv4
                index = packet.get_header_index(Ethernet)
                if index != -1:
                    del packet[index]
```

```

        packet.insert_header(0,new_eth)

        log_info(f"forwarding packet {packet} from {intf}")

        self.net.send_packet(intf,packet)

    else:

        self.append_waiting_list(packet = packet,next_hop = ip_header.dst,intf = intf)

else:

    #check if the packet needs ARP resolution

    # print(f"next_hop is {next_hop}")

    macaddr = self.arp_table.get(next_hop)

    if macaddr is not None:

        # print("macaddr is not None")

        new_eth = Ethernet()

        new_eth.dst= macaddr

        new_eth.src= intf.ethaddr

        new_eth.ethertype=EtherType.IPv4

        index = packet.get_header_index(Ethernet)

        if index != -1:

            del packet[index]

            packet.insert_header(0,new_eth)

            log_info(f"forwarding packet {packet} from {intf}")

            self.net.send_packet(intf,packet)

        else:

            # print("macaddr is None")

            self.append_waiting_list(packet = packet,next_hop = next_hop,intf = intf)

```

- forward_packet 函数用于将数据包转发到下一个节点。它先检查目的 IP 是否需要 ARP 解析,如果需要则发起 ARP 请求,并将数据包加入等待队列。
- 若下一跳地址已在 ARP 缓存中,则直接构造以太网帧并发送数据包。

■ ARP 请求与应答

```

def send_arp_request(self,dest_ip,interface):

    # construct ARP request packet

    arp_request=Arp()

    arp_request.operation=ArpOperation.Request

    arp_request.senderhwaddr=interface.ethaddr

    arp_request.senderprotoaddr=interface.ipaddr

    arp_request.targethwaddr='ff:ff:ff:ff:ff:ff'

    arp_request._targetprotoaddr=dest_ip

    # print("successfully create the arp header!")

    # create ethernet packet with ARP request

    ether=Ethernet()

    ether.dst='ff:ff:ff:ff:ff:ff'

    ether.src=interface.ethaddr

    ether.ethertype=EtherType.ARP

    # print("successfully create the ethernet header!")

```

```

# send ARP request
packet = ether + arp_request

# print(type(interface))

# print(type(packet))

# debugger()

log_info(f"Sending ARP request {packet}from {interface.name}")

self.net.send_packet(interface.name,packet)

print("successfully send the arp request!\n")

```

- 当路由器需要获取目标设备的 MAC 地址时，会发出 ARP 请求。该函数构造 ARP 请求并通过相应接口广播出去。

```

def send_arp_reply(self,ethdst,targethwaddr,targetprotoaddr,senderprotoaddr,senderhwaddr,interface):
    # construct ARP reply packet
    arp_reply = Arp()
    arp_reply.operation=ArpOperation.Reply
    arp_reply.senderhwaddr=senderhwaddr
    arp_reply.senderprotoaddr=senderprotoaddr
    arp_reply.targethwaddr=targethwaddr
    arp_reply._targetprotoaddr =targetprotoaddr
    # print("successfully create the arp header!")
    # create ethernet packet with ARP request
    ether=Ethernet()
    ether.dst=ethdst
    ether.src=interface.ethaddr
    ether.ethertype=EtherType.ARP
    packet = ether+arp_reply
    # print("successfully create the ethernet header!")
    # send ARP request
    # print(type(interface))
    # print(type(packet))
    # debugger()
    log_info(f"Sending ARP reply {packet}from {interface.name}")
    self.net.send_packet(interface.name,packet)
    print("successfully send the arp reply!\n")

```

- 当路由器收到 ARP 请求时，若目标 IP 地址是路由器接口的 IP，它将发送 ARP 回复，告诉请求方自己的 MAC 地址。

```

def update_arp_cache(self,ipaddr,macaddr):
    self.arp_table[ipaddr] = macaddr # update cache arp table
    print('Cache arp table:')
    print("{:<18}{:<18}".format("IP address","MAC address"))
    for key,value in self.arp_table.items():
        print("{:<18}{:<18}".format(str(key),str(value)))
    print('\n')
    # print(self.waiting_list)
    #send the packets waiting the corresponding arp reply

```

```

if self.waiting_list.get(ipaddr) is not None:
    queue = self.waiting_list[ipaddr]["queue"]
    interface = self.waiting_list[ipaddr]["interface"]
    new_eth = Ethernet()
    new_eth.dst= macaddr
    new_eth.src= interface.ethaddr
    new_eth.ethertype=EtherType.IPv4
    # print(queue)
    for pkt in queue:
        index = pkt.get_header_index(Ethernet)
        if index != -1:
            del pkt[index]
        pkt.insert_header(0,new_eth)
        log_info(f"sending the packets in waiting queue {pkt} from {interface}")
        self.net.send_packet(interface,pkt)
        print("successfully sended the packet in waiting queue!\n")
    self.waiting_list.pop(ipaddr)

```

- 该函数更新路由器的 ARP 缓存，并处理在 ARP 缓存中等待解析的包。当 ARP 解析完成后，缓存中的数据包会被重新封装以太网头并发送。

■ 处理接收的数据包

```

def handle_packet(self, recv: switchyard.llnetbase.ReceivedPacket):
    timestamp, ifaceName, packet = recv
    log_info(f"receiive packet {packet} from {ifaceName}\n")
    incoming_intf = self.net.interface_by_name(ifaceName)
    eth = packet.get_header(Ethernet)
    if eth.dst != "ff:ff:ff:ff:ff:ff" and eth.dst != incoming_intf.ethaddr:
        return # drop out the packet
    type = eth.ethertype
    if type == EtherType.ARP:
        arp = packet.get_header(Arp)# when the ARP destination IP is held by a port of the router
        if arp.targetprotoaddr in self.intf_ipaddrs:
            if not (arp.operation==ArpOperation.Reply and eth.src=='ff:ff:ff:ff:ff:ff'):
                # if not(eth.src == 'ff:ff:ff:ff:ff:ff' and eth.dst!='ff:ff:ff:ff:ff:ff'):#not trick
                self.update_arp_cache(ipaddr=arp.senderprotoaddr,macaddr=arp.senderhwaddr)
            if arp.operation == ArpOperation.Request: #need to send arp reply
                target_intf = self.net.interface_by_ipaddr(arp.targetprotoaddr)
                self.send_arp_reply(ethdst =
eth.src,targethwaddr=arp.senderhwaddr,targetprotoaddr=arp.senderprotoaddr,senderprotoaddr =
target_intf.ipaddr,senderhwaddr = target_intf.ethaddr,interface=incoming_intf)
        elif type == EtherType.IPv4:
            ip_header = packet.get_header(IPv4)
            ip_header.ttl-=1
            if ip_header.dst in self.intf_ipaddrs:

```

```

        return # drop the packet
    elif self.match(ip_header.dst) is not None:# succeed matching the correct item
        self.forward_packet(packet)
    else:# there is no macth in the table
        return#drop the packet
else:
    return

```

- handle_packet 函数是路由器的核心处理逻辑。
- 如果收到的是 ARP 包，路由器将根据包的内容更新 ARP 缓存或发送 ARP 回复。
- 如果是 IPv4 数据包，路由器首先检查是否为发往自己接口的包。如果不是，则调用 forward_packet 进行转发处理。

■ 检查 ARP 请求的重试

```

def check(self):
    # print("check!")
    delete_list = []
    for key,value in self.waiting_list.copy().items():
        if time.time()-value["time"] > 1:
            if value["retries"]<5:
                next_hop = key
                intf = value["interface"]
                self.send_arp_request(dest_ip = next_hop,interface = intf)
                self.waiting_list[next_hop]["time"] = time.time()
                self.waiting_list[next_hop]["retries"] += 1
            else:
                delete_list.append(key)
    for key in delete_list:
        self.waiting_list.pop(key)

```

- check 函数负责检查等待队列中的 ARP 请求。如果一个请求超时并且尚未超过重试次数，则重新发送 ARP 请求。如果超过重试次数，路由器会丢弃该请求。

■ 启动与停止

```

def start(self):
    '''A running daemon of the router.
    Receive packets until the end of time.
    ...
    while True:
        try:
            recv = self.net.recv_packet(timeout=1.0)
        except NoPackets:
            self.check()
            continue

```

```

except Shutdown:
    break

    self.handle_packet(recv)

self.stop()

```

➤ start 函数是路由器的主循环，负责持续监听并处理接收到的数据包。路由器会在每次超时调用 check 检查 ARP 请求的状态。

③注意细节

■ Task2

➤ 构建转发表

转发表信息的两个来源

a. router 的 interface

通过 net.interfaces() 获取 router 的 interface 对象；

直接访问其 ipaddr 与 netmask 属性获取对应的 IP 地址以及网络掩码；

利用 Networkaddr 进行地址转换获得该 interface 对应的子网地址，并且直接获取其对应的前缀长度。

b. forwarding_table.txt 文件

读取文件获取对应的字符串信息，进行类型转换；

利用 Networkaddr 进行地址转换子网地址，并且直接获取其对应的前缀长度。

➤ 转发表数据结构→[{}, {}, {}]

将 netaddr (子网地址) | netmask (网络掩码) | next_hop (下一跳地址) | interface (对应接口) | prefixlen (前缀长度) 等相关信息以字典形式存储起来，以字典为元素构建列表，即转发表；

将转发表按照字典中的前缀长度按照从大到小进行排序，以方便后续查找。

➤ 最大前缀匹配

```
prefix = IPv4Address('172.16.0.0')
```

```
mask = IPv4Address('255.255.0.0')
```

```
destaddr = IPv4Address('172.16.23.55')
```

```
matches = (int(mask) & int(destaddr)) == int(prefix)
```

设计方法 match 进行前缀匹配；

由于转发表按照前缀长度进行排序，所以直接按顺序尝试匹配，一旦发现匹配项即可直接返回。

■ Task3

➤ 整体逻辑

收到 packet 进行对应处理，若主循环中未收到，则进行一个 check，检查等待队列中是否有 arp request 发出去超时了，且重试次数没有超过五次。

➤ 处理收到的 packet

a. 检查 Ethernet 首部

Ethernet 地址不是广播地址也不是收到数据帧的 interface 的 mac 地址，直接丢掉

b. 判断数据报类型

i. ARP 首部

若符合 FAQ 中的要求则更新对应的 ARP 表；

若是 request, 则发出对应的 arp reply (注意也同样需要查找对应的转发表项) 。

ii. IPv4 首部

若是 0.0.0.0, 那么 next_hop 就相当于 dest_ip, 说明已经到达了目的 ip 所在的子网 ;

若有匹配项, 就获取 next_hop ;

通过 next_hop 查找 arp table, 获取对应的 mac 地址 ;

若 arp table 中没有匹配项, 则发送 arp request 。

(2) 测试结果

```
(syenv) njucs@njucs-VirtualBox:~/workspace/lab-4-221275014$ swyard -t testcases/testscenario2.srpy myrouter.py
09:54:42 2024/10/17 INFO Starting test scenario testcases/testscenario2.srpy
```

```
njucs@njucs-VirtualBox: ~/workspace/lab-4-221275014
File Edit View Search Terminal Help
23 Router should send an ARP request for 10.10.50.250 on
    router-eth1
24 Router should try to receive a packet (ARP response), but
    then timeout
25 Router should send an ARP request for 10.10.50.250 on
    router-eth1
26 Router should try to receive a packet (ARP response), but
    then timeout
27 Router should send an ARP request for 10.10.50.250 on
    router-eth1
28 Router should try to receive a packet (ARP response), but
    then timeout
29 Router should send an ARP request for 10.10.50.250 on
    router-eth1
30 Router should try to receive a packet (ARP response), but
    then timeout
31 Router should try to receive a packet (ARP response), but
    then timeout

All tests passed!
```

```
File Edit View Search Terminal Help
(syenv) njucs@njucs-VirtualBox:~/workspace/lab-4-221275014$ swyard -t testcases/testscenario2_advanced.srpy myrouter.py
09:56:42 2024/10/17 INFO Starting test scenario testcases/testscenario2_advanced.srpy
10.10.0.1
255.255.0.0
```

```
njucs@njucs-VirtualBox: ~/workspace/lab-4-221275014
File Edit View Search Terminal Help
1201Ping request from 31.0.6.1 should arrive on eth6
1202Ping request from 31.0.6.1 should arrive on eth6
1203Ping request from 31.0.6.1 should arrive on eth6
1204Ping request from 31.0.6.1 should arrive on eth6
1205Router should not do anything
1206Bonus: V2FybWluZyB1cA==
1207Bonus: V2FybWVkJHVw
1208Bonus: V2h1dCBkYjB5SBob3B1IHQnIGZpbmQgaGVyZT8=

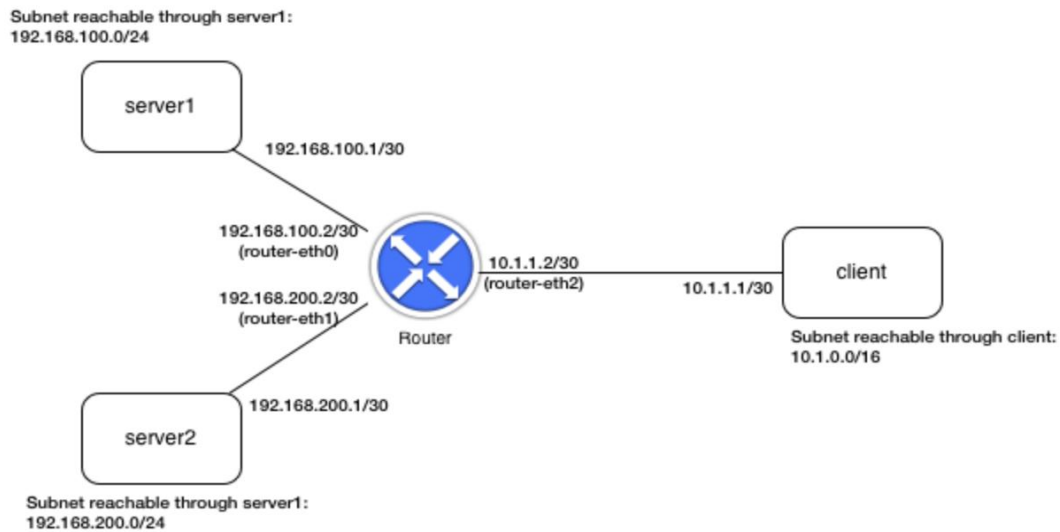
Failed:
  Bonus: SGFsZndheQ==
  Expected event: Timeout after 1.2s on a call to recv_packet

Pending (couldn't test because of prior failure):
1 Bonus: Tm90aGluJyBmb3IgeWEgdCcgZmluZCBoZXJlIQ==
2 Bonus: Q29uZ3JhdHMh

*****
Your code didn't crash, but a test failed.
*****
```

(3) 部署结果

拓扑结构如下：



打开终端，进入实验文件夹，`sudo python start mininet.py` 运行 mininet
→ 打开 router 的 xterm: `xterm router`
→ 激活虚拟环境，运行 router: `swyard myrouter.py`
→ `xterm server1` 打开 server1 的 xterm
→ 打开 wireshark, 在 eth0 端口抓包 `router wireshark i routereth0`
→ 在 server1 的 xterm 输入 `ping c2 192.168.200.1`
mininet 结果：

```
*** Starting 0 switches

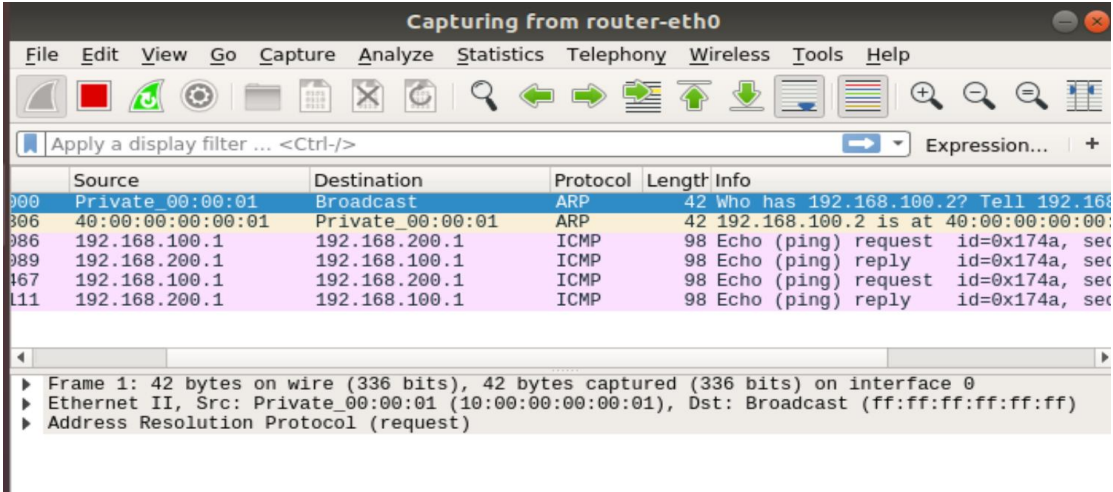
*** Starting CLI:
mininet> xterm router
mininet> xterm server1
mininet> router wireshark -i router-eth0
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runtime-root'
```

Server1 节点：

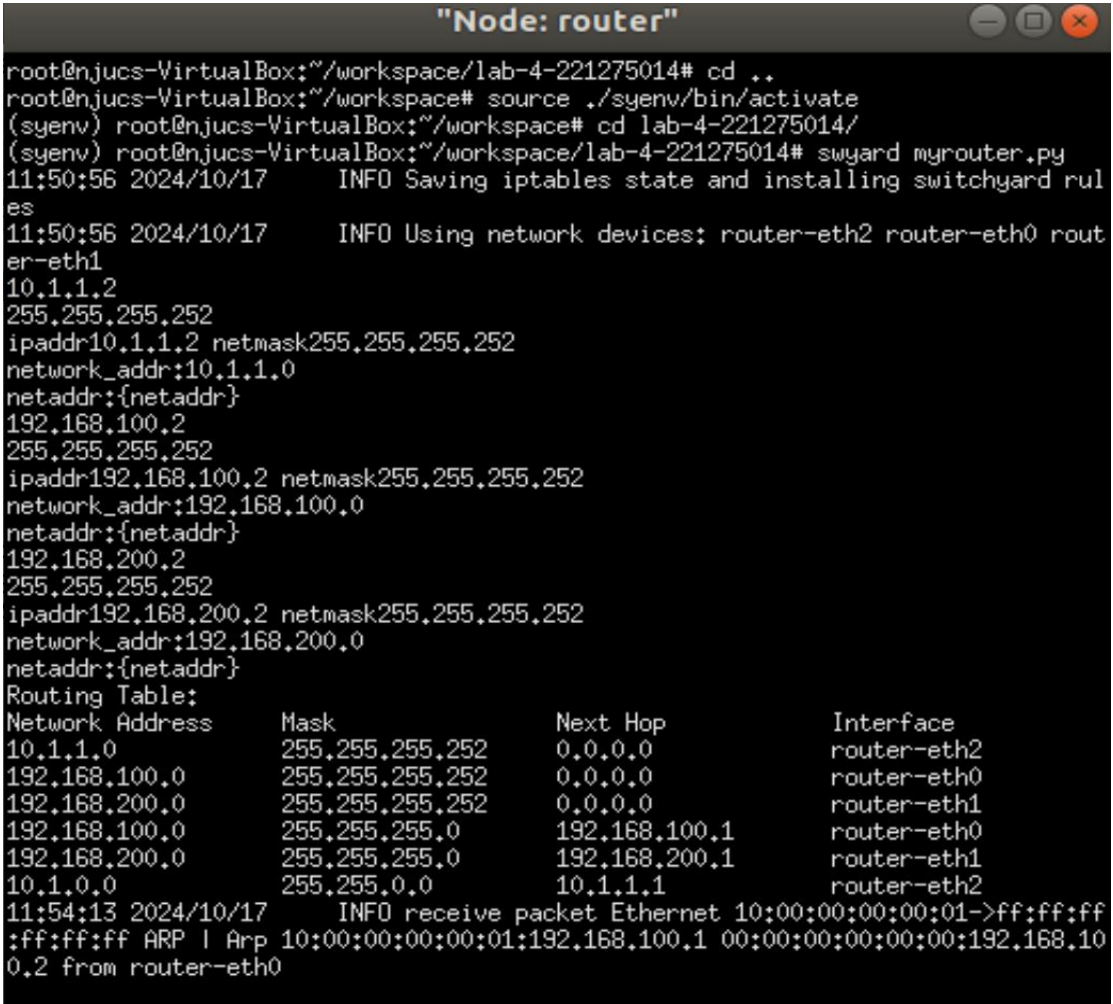
```
"Node: server1"
root@njucs-VirtualBox:~/workspace/lab-4-221275014# ping -c2 server2
ping: server2: Name or service not known
root@njucs-VirtualBox:~/workspace/lab-4-221275014# ping -c2 192.168.200.1
PING 192.168.200.1 (192.168.200.1) 56(84) bytes of data.
64 bytes from 192.168.200.1: icmp_seq=1 ttl=63 time=388 ms
64 bytes from 192.168.200.1: icmp_seq=2 ttl=63 time=122 ms

--- 192.168.200.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 122.343/255.302/388.262/132.960 ms
root@njucs-VirtualBox:~/workspace/lab-4-221275014#
```

Wireshark: router-eth0:



router 终端:



```
"Node: router"

Cache arp table:
IP address      MAC address
192.168.100.1   10:00:00:00:00:01

11:54:13 2024/10/17      INFO Sending ARP reply Ethernet 40:00:00:00:00:01->10:00:00:00:00:01 ARP | Arp 40:00:00:00:00:01:192.168.100.2 10:00:00:00:00:01:192.168.100.1from router-eth0
successfully sended the arp reply!

11:54:13 2024/10/17      INFO receive packet Ethernet 10:00:00:00:00:01->40:00:00:00:00:01 IP | IPv4 192.168.100.1->192.168.200.1 ICMP | ICMP EchoRequest 5962 1 (56 data bytes) from router-eth0

match successfully!
next_hop:0.0.0.0,intf:router-eth1 mac:40:00:00:00:00:02 ip:192.168.200.2/30
waiting list hasn't got the item!
successfully created the queue!
11:54:13 2024/10/17      INFO Sending ARP request Ethernet 40:00:00:00:00:02->ff:ff:ff:ff:ff:ff ARP | Arp 40:00:00:00:00:02:192.168.200.2 ff:ff:ff:ff:ff:ff:192.168.200.1from router-eth1
successfully sended the arp request!

11:54:14 2024/10/17      INFO receive packet Ethernet 20:00:00:00:00:01->40:00:00:00:00:02 ARP | Arp 20:00:00:00:00:01:192.168.200.1 40:00:00:00:00:02:192.168.200.2 from router-eth1

Cache arp table:
IP address      MAC address
192.168.100.1   10:00:00:00:00:01
192.168.200.1   20:00:00:00:00:01

11:54:14 2024/10/17      INFO sending the packets in waiting queue Ethernet 40:00:00:00:00:02->20:00:00:00:00:01 IP | IPv4 192.168.100.1->192.168.200.1 ICMP | IC

"Node: router"

11:54:14 2024/10/17      INFO sending the packets in waiting queue Ethernet 40:00:00:00:00:02->20:00:00:00:00:01 IP | IPv4 192.168.100.1->192.168.200.1 ICMP | IC
MP EchoRequest 5962 1 (56 data bytes) from router-eth1 mac:40:00:00:00:00:02 ip:192.168.200.2/30
successfully sended the packet in waiting queue!

11:54:14 2024/10/17      INFO receive packet Ethernet 20:00:00:00:00:01->40:00:00:00:00:02 IP | IPv4 192.168.200.1->192.168.100.1 ICMP | ICMP EchoReply 5962 1 (56 data bytes) from router-eth1

match successfully!
next_hop:0.0.0.0,intf:router-eth0 mac:40:00:00:00:00:01 ip:192.168.100.2/30
11:54:14 2024/10/17      INFO forwarding packet Ethernet 40:00:00:00:00:01->10:00:00:00:00:01 IP | IPv4 192.168.200.1->192.168.100.1 ICMP | ICMP EchoReply 5962 1 (56 data bytes) from router-eth0 mac:40:00:00:00:00:01 ip:192.168.100.2/30
11:54:14 2024/10/17      INFO receive packet Ethernet 10:00:00:00:00:01->40:00:00:00:00:01 IP | IPv4 192.168.100.1->192.168.200.1 ICMP | ICMP EchoRequest 5962 2 (56 data bytes) from router-eth0

match successfully!
next_hop:0.0.0.0,intf:router-eth1 mac:40:00:00:00:00:02 ip:192.168.200.2/30
11:54:14 2024/10/17      INFO forwarding packet Ethernet 40:00:00:00:00:02->20:00:00:00:00:01 IP | IPv4 192.168.100.1->192.168.200.1 ICMP | ICMP EchoRequest 5962 2 (56 data bytes) from router-eth1 mac:40:00:00:00:00:02 ip:192.168.200.2/30
11:54:14 2024/10/17      INFO receive packet Ethernet 20:00:00:00:00:01->40:00:00:00:00:02 IP | IPv4 192.168.200.1->192.168.100.1 ICMP | ICMP EchoReply 5962 2 (56 data bytes) from router-eth1

match successfully!
next_hop:0.0.0.0,intf:router-eth0 mac:40:00:00:00:00:01 ip:192.168.100.2/30
11:54:14 2024/10/17      INFO forwarding packet Ethernet 40:00:00:00:00:01->10:00:00:00:00:01 IP | IPv4 192.168.200.1->192.168.100.1 ICMP | ICMP EchoReply 5962 2 (56 data bytes) from router-eth0 mac:40:00:00:00:00:01 ip:192.168.100.2/30
11:54:19 2024/10/17      INFO receive packet Ethernet 20:00:00:00:00:01->40:00:00:00:00:02 ARP | Arp 20:00:00:00:00:01:192.168.200.1 00:00:00:00:00:00:192.168.200.2 from router-eth1

Cache arp table:
IP address      MAC address
192.168.100.1   10:00:00:00:00:01
192.168.200.1   20:00:00:00:00:01

11:54:19 2024/10/17      INFO Sending ARP reply Ethernet 40:00:00:00:00:02->20:00:00:00:00:01 ARP | Arp 40:00:00:00:00:02:192.168.200.2 20:00:00:00:00:01:192.168.200.1from router-eth1
successfully sended the arp reply!
```

四、总结与感想

学习了路由器更多功能及逻辑；学习了路由器如何构建转发表、匹配目标地址、进行 ARP 查询、转发数据包

出现问题：

1. swyard 手册相关问题

interface 没有 ipaddrs 在文件中发现更新为了 ipaddr, interface 的 netmask 属性是直接查看源代码发现的；

pkt[Ethernet]=Ethernet() 不行？（手册中写法，但实际运行过程中会报错，index must be integar）；

但 pkt[0]=Ethernet() 可以用。

2. 类型转换相关问题

eth.src/eth.dst/..... 等返回的是 IPv4Address 的对象，而不是字符串；

print 出来看不出区别，需要用 type() 进行类型检查；

next_hop 起初没有进行类型转换，直接以字符串的形式记入字典中。后直接读取 next_hop 并用其构造 packet 时报错 object 'str' has no attribute 'packed'。甚至 print 出来也看不出区别。直接调用 create_arp_request() 接口可能会自动完成类型检查和转换，可以避免错误。不过后面通过类型检查避免了错误。

3. 对 ARP 的处理

注意 arp 的欺诈行为（猜测是 eth 的 src 是广播地址，但 dst 是接口 macaddr）；

注意回应 arp request（区分 packet 中的 senderprotoaddr 和 senderhwaddr 不是来自其发出的接口，接口不是 target 接口，而是收到 request 的接口；以及 ethernet 的首部填写内容也应该只与 arp request 的 ethernet 首部有关）。